

VOLUMEN III · LIBRO 3 DE 5

BMAD Plan de Sesiones

El calendario operativo: cómo se llevan a la realidad las cuatro
fases

Seis sesiones operativas para implantar el método.
Agendas, participantes, decisiones de equipo y plantillas.
Propuesta de adopción de LIDR sobre el flujo BMAD.

BMAD Method · CIS · TEA · BMB · Core

2026-05-07 · LIDR.co

0 Bienvenida

Este es el tercer libro de la biblioteca. A diferencia de los otros, no documenta lo que el método BMAD prescribe, sino la **propuesta de LIDR para llevarlo a un equipo real**: cómo distribuimos las cuatro fases en sesiones operativas, qué se decide en cada una, cuánto suelen durar y quién participa.

VISIÓN LIDR

El método oficial no prescribe sesiones: deja la planificación a cada equipo. El calendario S0–S6 que estructura este libro, los roles de BMAD Master y BMAD Driver, los toolkits asíncronos post-S2 y post-S6, y el conjunto de tres decisiones operativas obligatorias en S4 son framing nuestro, no parte de BMAD. El lector verá la insignia "Visión LIDR" en cada framing conceptual y "Recomendación LIDR" en cada sugerencia de práctica.

0.1 Cómo está organizado el libro

Sección	Contenido	Cuándo la usas
Sec. 1 Convenciones	Módulos, insignias, anatomía de la ficha de sesión.	Hojear la primera vez para entender el formato.
Sec. 2 Visión general	Resumen de las siete sesiones, calendario en diagrama, mapeo a fases del proyecto y roles fijos del equipo.	Para entender el calendario completo antes del detalle.
Sec. 3 Sesión preliminar S0	Aprendizaje de testing antes de adoptar TEA. Solo si aplica.	En equipos sin disciplina previa de testing.
Sec. 4–Sec. 9 Sesiones S1 a S6	Una ficha por sesión con resuelve, participantes, requisitos previos, agenda detallada, output y siguiente paso.	Para preparar y facilitar cada sesión.
Sec. 10 Plantillas operativas	Plantilla del archivo <code>project-context.md</code> , checklist pre-sesión y checklist de cierre.	Como recursos imprimibles para uso del equipo.
Sec. 11 Verificación y fuentes	Versión BMAD verificada, repositorios oficiales consultados.	Para auditar la vigencia del libro.

0.2 Sesión, fase y skill: cómo se distinguen

Concepto	Qué es	Dónde se documenta
Skill	Workflow ejecutable que produce un artefacto. Se invoca con <code>/comando</code> .	Libro 4 (Catálogo de Skills) — ficha completa por skill.
Fase	Etapas secuenciales del flujo del proyecto. Hay cuatro: análisis, planificación, solución, implementación.	Libro 2 (Flujos de Proyecto) — cómo se enlazan las skills en cada fase.

↓ continúa en la página siguiente

1 Convenciones

1.1 Punto de color del módulo

- **BMM** · flujo principal
- **TEA** · testing
- **CIS** · pensamiento creativo
- **BMB** · extensión del método
- **Core** · utilidades transversales

1.2 Insignias de obligatoriedad por participante

REQ obligatoria para el objetivo de la sesión · **REC** recomendada fuertemente · **OPC** opcional, la sesión avanza sin ese rol pero pierde valor.

1.3 Insignias de aporte LIDR

VISIÓN LIDR framing conceptual nuestro · **RECOMENDACIÓN LIDR** sugerencia operativa basada en nuestra experiencia.

1.4 Anatomía de la ficha de sesión

Cada sesión del Sec. 3 al Sec. 9 sigue siempre el mismo orden, para que el lector localice la información en el mismo sitio sin tener que leerla entera:

1. **Cabecera** — identificador (S0...S6), título y duración recomendada.
2. **Resuelve** — el problema que aborda en una frase.
3. **Cuándo se hace** — qué precede a la sesión.
4. **Participantes** — roles del equipo con su grado de obligatoriedad.
5. **Requisitos previos** — artefactos o estado que debe haber antes.
6. **Agenda** — pasos concretos con duración y skills BMAD que se invocan.
7. **Output** — qué se llevan los participantes al salir.
8. **Siguiente paso** — qué viene después en el calendario.

2 Visión general del calendario

2.1 Las siete sesiones de un vistazo

↓ continúa en la página siguiente

Sesión	Objetivo	Duración	Cubre del flujo	Output principal
S0	Aprendizaje de testing — opcional, solo si se adopta TEA.	14–28 h en 1–2 semanas (autodirigido).	Pre-adopción.	Equipo formado en disciplina de testing.
S1	Análisis del producto: brief, PRD y UX Design.	3 h.	Fase 1 + Fase 2 comprimidas.	PRD validado con NFR cuantificados + UX Design.
S2	Arquitectura técnica y arranque de la estrategia de testing.	3 h + toolkit asíncrono.	Fase 3 primer tramo.	Arquitectura + Test Design + Framework instalado + pipeline de integración continua.
S3	Plan final + arranque de la implementación.	3 h 30 min.	Fase 3 cierre + Fase 4 inicio.	Épicas + readiness aprobado + sprint planificado + primera historia con tests ATDD ¹ en rojo.
S4	Convenciones del equipo + ajustes BMAD en vivo.	2–3 h.	Transición operativa a la implementación.	Sección "Ops Conventions" en <code>project-context.md</code> commiteada.
S5	Ciclo iterativo por historia.	Continuo, post-S4.	Fase 4 en modo continuo.	Pull requests mergeados con tests automatizados y revisión.
S6	Cierre formal de la primera épica + toolkit asíncrono NFR Check.	2–3 h.	Cierre TEA acompañado.	Test Review + Trazabilidad + Retrospectiva + autonomía confirmada.

RECOMENDACIÓN LIDR

En proyectos típicos, S1 a S4 ocupan unas **11–13 h** distribuidas en una o dos semanas; el equipo entra a S5 con todo lo necesario para operar. Tras S6 (otras 2–3 h al cerrar la primera épica), el equipo opera autónomo. La cifra es nuestra: el método no fija duraciones, depende del tamaño y la experiencia del equipo.

2.2 Calendario en una imagen

El siguiente diagrama resume la secuencia. Las flechas continuas marcan dependencias estrictas; la flecha punteada indica que S0 es opcional.

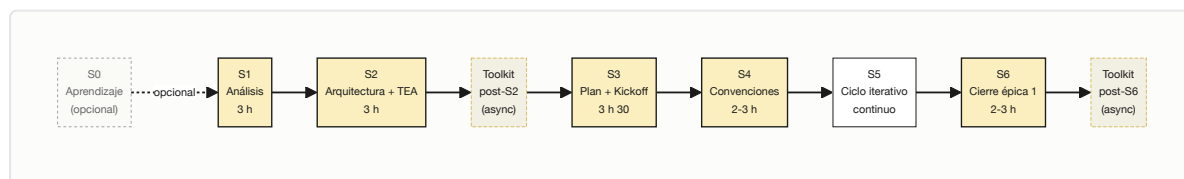


Figura 2.1 — Calendario de adopción LIDR sobre el flujo BMAD.

2.3 Mapeo a fases del proyecto

Cada sesión cubre una porción concreta del flujo descrito en el Libro 2.

↓ continúa en la página siguiente

1. ATDD: Acceptance Test-Driven Development. Variante de TDD donde los tests se derivan directamente de los criterios de aceptación de la historia y se escriben antes del código de producción. Generan la fase roja del ciclo red-green-refactor.

Sesión	Tramo del flujo del Libro 2	Por qué se agrupa así
S1	Fase 1 + Fase 2 comprimidas (research, brief, PRD, UX, NFR setup).	Permite cerrar planificación de producto en una sola sesión cuando el equipo es pequeño.
S2	Fase 3 primer tramo: arquitectura + bloque TEA de arranque (test design, framework, pipeline).	El bloque TEA depende del stack que la arquitectura fija; se ejecuta en la misma sesión para que las decisiones queden alineadas.
S3	Cierre de Fase 3 (épicas, readiness) + inicio de Fase 4 (sprint planning, primera historia, ATDD).	Encadena planificación con kickoff para que el equipo salga con la primera historia lista para implementar.
S4	Transición operativa a Fase 4: decisiones del equipo y ajustes en BMAD.	El método no prescribe tool ni branch strategy: estas decisiones se toman aquí y se persisten para que las skills downstream las respeten.
S5	Fase 4 en modo continuo — ciclo iterativo por historia.	No es sesión facilitada: es el modo de operación del equipo entre sesiones.
S6	Cierre formal de la primera épica: test-review → trace → retrospective .	El primer cierre se facilita con red de seguridad; en épicas posteriores el equipo lo ejecuta solo.

2.4 Roles fijos en todas las sesiones

Más allá de los roles que cubren cada fase del proyecto (PM, PO, Tech Lead, UX, QA, Dev, SM, DevOps), durante la adopción del método LIDR define dos roles permanentes que aparecen en todas las sesiones facilitadas.

VISIÓN LIDR **BMAD Master** y **BMAD Driver** son roles humanos que LIDR define para acompañar la adopción. Ninguno existe en el método como agente o skill. Su función es asegurar que el proceso se ejecute correctamente y que la herramienta no sea un obstáculo para el equipo de producto.

Rol	Presencia	Responsabilidad	Perfil típico
BMAD Master	S1 inicio y cierre, S2 inicio, S3 inicio y cierre, S4 toda la sesión, S6 cierre con Q&A.	Facilitador de proceso. Asegura que el método se ejecute correctamente; no valida contenido técnico ni de negocio. En S4 implementa los ajustes de BMAD en vivo.	Consultor con dominio del método, o líder de proceso del cliente con experiencia previa en adopciones BMAD. En equipos maduros, puede coincidir con el Tech Lead.
BMAD Driver	Toda la sesión, en todas las sesiones facilitadas.	Opera el agente de IA y ejecuta todas las skills BMAD durante la sesión. Traduce el input del equipo de producto en comandos.	Tech Lead o desarrollador senior con experiencia en el agente que se está usando.

DINÁMICA DE LAS SESIONES

El equipo de producto aporta contenido y decisiones de negocio. El Driver traduce ese input en invocaciones de skills. El Master verifica que los pasos del proceso se completen y que los artefactos producidos sean coherentes con el método. Esta separación de responsabilidades permite que la sesión avance sin que nadie tenga que dominar a la vez producto, técnica y BMAD.

2.5 Resto de roles del equipo

Los demás roles aparecen según la sesión. Su presencia y obligatoriedad se detalla en cada ficha. Para la lista completa de responsabilidades por rol, consulta el Libro 2 (BMAD Flujos de Proyecto) sección 5.

3 Sesión preliminar S0 — Aprendizaje

S0 Aprendizaje de testing — opcional, autodirigido

14–28 h · 1–2 semanas

RESUELVE: Forma al equipo en testing antes de que empiece a usar TEA en sesiones reales. Es la red de seguridad para equipos sin disciplina previa.

CUÁNDO SE HACE

Antes de S2 (que es donde aparecen las primeras skills TEA). Solo si el equipo no domina conceptos base de testing.

REQUISITOS PREVIOS

Acceso al agente de IA con la skill `/bmad-teach-me-testing` instalada. Sin dependencia de PRD ni arquitectura.

PARTICIPANTES

Cualquier integrante del equipo que vaya a tocar TEA: Dev, QA y, opcionalmente, Tech Lead. No requiere facilitación.

OUTPUT

Equipo formado en disciplina de testing y certificación de finalización del programa.

Detalle pleno: esta sesión es un envoltorio del workflow `/bmad-teach-me-testing`. El detalle del programa, sus 7 micro-sesiones académicas y los criterios de progreso viven en el Libro 4 (Catálogo de Skills) ficha 01.

¿CUÁNDO CONVIENE S0?

Si el equipo ya escribe tests automatizados con regularidad, S0 es prescindible. Si los tests existentes son escasos, frágiles o se generan en bloque al final del sprint, conviene activarla. La inversión inicial reduce drásticamente el tiempo perdido en S2 y S3 explicando conceptos básicos.

4 Sesión S1 — Análisis del Producto

S1 Análisis del producto — brief, PRD y UX Design

3 h · sesión 1

RESUELVE: Convierte una idea de producto en un PRD² con NFR³ cuantificados y un UX Design listo para arquitectura, todo en una sola sesión.

CUÁNDO SE HACE

Primera sesión del proyecto. Sin prerequisite BMAD; idealmente con research previo asíncrono.

REQUISITOS PREVIOS

- PO y UX traen documentación recaudada (idea, brief si existe).
- Driver prepara entorno BMAD.
- Opcional pre-sesión: `/bmad-market-research`, `/bmad-domain-research`, `/bmad-technical-research`

3. PRD: *Product Requirements Document*. Documento que describe lo que el producto debe hacer (requisitos funcionales) y cómo de bien debe hacerlo (requisitos no funcionales). Es el contrato entre producto y equipo técnico.

3. NFR: *Non-Functional Requirement*. Requisito sobre cómo de bien debe comportarse el sistema (rendimiento, seguridad, fiabilidad, escalabilidad), en contraposición a los requisitos funcionales que describen qué debe hacer.

ejecutados en asíncrono.

PARTICIPANTES

- BMAD Master **REQ** — inicio y cierre.
- BMAD Driver **REQ** — toda la sesión.
- Product Owner **REQ** — toda la sesión.
- UX Designer **REQ** — primeras 2 h 30.
- Tech Lead **REQ** — última media hora (NFR setup).
- DevOps **OPC** — última media hora.

OUTPUT

Brief + PRD validado con NFR cuantificados + UX Design.

AGENDA

1. **Apertura del Master** — alinea qué se produce, qué necesita cada rol y decide si activar workflows del módulo creativo. ~10 min.
2. **/bmad-brainstorming** ● opcional, si la idea no está cristalizada. PO + UX. Alternativas: **/bmad-cis-design-thinking**, **/bmad-cis-storytelling**. ~30–45 min.
3. **/bmad-product-brief** ● — PO produce el brief. ~30 min.
4. **/bmad-create-prd** ● — PO + UX. Cada requisito funcional debe ser testeable (formato Given/When/Then⁴). ~45–60 min.
5. **/bmad-create-ux-design** ● — UX. Tras este paso, UX se retira y entran TL y DevOps. ~30 min.
6. **/bmad-testarch-nfr en modo setup** ● — PO + TL + DevOps. Fija umbrales cuantificables (latencia P95⁵, RPS⁶, SLO⁷). ~25 min.
7. **Cierre del Master** — verificación de proceso. ~10 min.
8. **Asíncrono post-sesión** — **/bmad-validate-prd** ejecutado por el Driver con un LLM distinto al que produjo el PRD. ~60 min en otra ventana.

Siguiente paso: S2 — la arquitectura y el bloque TEA necesitan el PRD con NFR cuantificados.

5 Sesión S2 — Arquitectura y arranque TEA

S2

Arquitectura técnica y estrategia de testing

3 h · sesión 2

RESUELVE: Diseña la arquitectura sobre el PRD y deja lista la estrategia de testing — plan basado en riesgo, framework instalado y pipeline de integración continua — antes de que aparezca la primera historia.

7. Given/When/Then: estructura de criterios de aceptación que describe un escenario en tres partes — dado un contexto inicial (Given), cuando ocurre un evento (When), entonces se espera un resultado (Then). Es el formato estándar de BDD.

7. P95: percentil 95 de latencia. Significa que el 95 % de las peticiones responden en un tiempo igual o inferior al valor declarado. Se usa porque la media oculta los casos lentos y la P50 (mediana) ignora la cola larga; P95 captura el comportamiento real para el 95 % de los usuarios.

7. RPS: *requests per second*. Métrica de carga que indica cuántas peticiones por segundo soporta un sistema. Se usa para fijar los umbrales no funcionales del PRD junto con la latencia P95.

7. SLO: *Service Level Objective*. Objetivo cuantificable de calidad de servicio que el equipo se compromete a cumplir — por ejemplo, "el 99,9 % de las peticiones responde en menos de 200 ms".

CUÁNDO SE HACE

Tras S1 cerrada con PRD y NFR cuantificados.

PARTICIPANTES

- BMAD Master **REQ** — solo apertura.
- BMAD Driver **REQ** — toda la sesión.
- Product Owner **REQ** — primeros 30 min para handoff de contexto.
- Architect / Tech Lead **REQ** — toda la sesión.
- QA **REQ** — última hora y media.
- DevOps **REQ** — última hora y media.

REQUISITOS PREVIOS

- S1 completa con PRD validado y NFR fijados.
- `/bmad-validate-prd` ejecutado en asíncrono entre S1 y S2.

OUTPUT

Arquitectura + plan de testing basado en riesgo + framework instalado + pipeline configurada.

AGENDA

1. **Apertura del Master** y handoff del PO al Architect del contexto del PRD. ~30 min.
2. `/bmad-create-architecture` ● — el Architect define stack, estructura y restricciones. ~60–90 min.
3. *Incorporación de QA y DevOps.*
4. `/bmad-testarch-test-design` ● — planificación basada en riesgo. Architect + QA. ~30–45 min.
5. `/bmad-testarch-framework` ● — scaffolding del framework. Architect + QA. ~25–35 min.
6. `/bmad-testarch-ci` ● — pipeline con gates de calidad. Architect + QA + DevOps. ~20–30 min.
7. **Gate de testabilidad post-arquitectura** — aplicar el ADR⁸ Quality Readiness Checklist (29 criterios en 8 categorías). El detalle vive en el Libro 5 (Protocolo TEA). ~10 min.

DEPENDENCIAS ERICTAS QUE ESTA SESIÓN DEJA PREPARADAS

El framework debe quedar instalado **antes** de S3, porque sin él los tests ATDD no son ejecutables. La pipeline debe estar configurada **antes** del primer merge en S5, no antes; pero se hace aquí mientras DevOps está presente. Saltarse cualquiera de las dos genera retrabajo difícil de localizar.

Siguiente paso: S3 — épicas, sprint planning y kickoff de la implementación. Entre medias, toolkit asíncrono si aparece desviación entre PRD y arquitectura.

TOOLKIT POST-S2 · ASÍNCRONO · SIN SESIÓN NUEVA

COURSE-CORRECTION TRAS S2

Si tras S2 aparece desviación entre el PRD y la arquitectura o la UX, no es necesario convocar sesión nueva. Hay tres rutas asíncronas según la magnitud del cambio:

- **Cambio menor de alcance, FR o NFR** → `/bmad-edit-prd` . Driver + PO. ~45–90 min.
- **Cambio estructural con impacto en alcance** → `/bmad-correct-course` . Driver + PO + Tech Lead. ~60–90 min.
- **Conflicto entre producto y arquitectura** → `/bmad-cis-problem-solving` . Driver + PO + Architect (+ UX si afecta al flujo de usuario). ~60 min.

Activación opcional. Solo escalar a llamada corta de 45–60 min si aparece desacuerdo no resoluble por chat.

VISIÓN LIDR

El método no contempla un "toolkit asíncrono" entre sesiones: cada skill simplemente se invoca cuando hace falta. Lo nombramos así porque, en una adopción tipo, los huecos descubiertos tras S2 no merecen una sesión completa pero sí necesitan

8. ADR: *Architecture Decision Record*. Documento corto que captura una decisión técnica (qué se eligió, qué se descartó y por qué), con sus trade-offs.

6 Sesión S3 — Plan Final + Kickoff

S3 Plan final y kickoff de la implementación

3 h 30 · sesión 3

RESUELVE: Cierra la planificación con épicas y readiness, hace el handoff al Scrum Master, planifica el primer sprint y arranca la primera historia con sus tests ATDD en rojo. Todo en un único flujo continuo.

CUÁNDO SE HACE

Tras S2 con arquitectura, framework instalado y pipeline operativa. Toolkit post-S2 aplicado si hubo desviación.

PARTICIPANTES

- BMAD Master **REQ** — inicio y cierre.
- BMAD Driver **REQ** — toda la sesión.
- Product Owner **REQ** — toda la sesión.
- Tech Lead **REQ** — toda la sesión.
- QA **REQ** — toda la sesión.
- Scrum Master **OPC** — segunda mitad. Si no hay SM, lo absorbe PO o TL.

REQUISITOS PREVIOS

- PRD + arquitectura completos.
- Test framework instalado.
- Pipeline configurada.

OUTPUT

Épicas e historias + readiness aprobado + sprint planificado + primera historia con contexto + tests ATDD en rojo.

AGENDA — CADENA ESTRICTA, NO PARALELIZABLE

1. **Apertura del Master** — explica los dos tramos (planificación y kickoff) y el gate de readiness intermedio. ~10 min.
2. **/bmad-create-epics-and-stories** ● — PO + Tech Lead. ~45–60 min.
3. **QA revisa criterios de aceptación** — verifica que cada criterio sea testeable en formato Given/When/Then. ~20 min.
4. **/bmad-check-implementation-readiness** ● — gate consume todo lo producido en S1, S2 y el toolkit. ~60 min.
5. **Handoff formal del PO al SM** — o auto-handoff si PO y SM coinciden. ~10 min.
6. **/bmad-sprint-planning** ● — SM o PO con el equipo técnico. ~15–20 min.
7. **/bmad-create-story** ● — primera historia con todo el contexto. Tech Lead + QA. ~40–60 min.
8. **/bmad-testarch-atdd** ● — fase roja de la primera historia. QA + Tech Lead. ~40–60 min.
9. **Cierre del Master** — verifica artefactos y alinea hacia S4. ~10 min.

Siguiente paso: S4 — convenciones del equipo y ajustes en BMAD para que las skills posteriores las respeten.

7 Sesión S4 — Convenciones del Equipo

S4 Convenciones del equipo + ajustes BMAD en vivo

2-3 h · sesión 4

RESUELVE: Cierra las decisiones operativas que el método no prescribe (herramienta de backlog, estrategia de ramas, flujo de estados, multi-agente) y las persiste en BMAD para que las skills posteriores las respeten automáticamente.

CUÁNDO SE HACE

Tras S3 completa, con épicas, sprint planificado y primera historia con ATDD en rojo. El equipo decide convenciones con artefactos concretos en mano, no en abstracto.

PARTICIPANTES

- BMAD Master **REQ** — toda la sesión, intermediario e implementador.
- Product Owner **REQ**.
- Tech Lead **REQ**.
- DevOps **REQ**.
- QA **REQ**.
- Scrum Master **OPC**.

REQUISITOS PREVIOS

- S3 cerrada con artefactos concretos.
- Driver con permisos para editar `project-context.md` y, si aplica, `.gitignore` y `CLAUDE.md`.

OUTPUT

Sección "Ops Conventions" en `project-context.md` commiteada al repositorio + ajustes en hooks o plantillas + (opcional) versión narrativa para onboarding.

AGENDA — DOS BLOQUES

1. **Bloque 1 — deliberación de las decisiones** (~1-1 h 30, +15-20 min si se aborda la decisión 4). Las tres decisiones obligatorias se toman en cualquier orden — son independientes — y la decisión 4 al final si aplica.
2. **Bloque 2 — ajustes BMAD en vivo** (~1-1 h 30). El Master ejecuta `/bmad-generate-project-context` si el archivo no existe, añade la sección "Ops Conventions" con las decisiones, ajusta hooks o plantillas y, si aplica decisión 4, edita `.worktreeinclude`, `.gitignore` y `CLAUDE.md`. Equipo presente para validar. Commit al repositorio antes de cerrar.

Siguiente paso: S5 — ciclo iterativo. Las skills downstream (`create-story`, `dev-story`, `code-review`, varias TEA) leen `project-context.md` como contexto y respetan automáticamente las convenciones acordadas.

RECOMENDACIÓN LIDR

Las tres decisiones operativas obligatorias y la opcional son nuestras. El método no las prescribe ni las exige; las hemos identificado como las que más fricción generan cuando un equipo arranca implementación sin acordarlas. La opcional (decisión 4) solo aplica si el equipo trabaja con el agente Claude Code; en cualquier otro contexto se omite.

7.1 Las cuatro decisiones operativas

D1 Herramienta de backlog externo

RESUELVE	Dónde trackea el equipo el backlog día a día. Sin acuerdo, las historias quedan en BMAD pero el negocio no las ve.
OPCIONES	Jira, Linear, Notion, GitHub Projects, Asana, Shortcut. El método no prescribe cuál; sirve cualquiera si se usa con consistencia.
QUIÉN DECIDE	El Product Owner. La estructura recomendada es <i>épica padre</i> + <i>historias hija</i> : cada épica como ticket parent al inicio, historias como children en pipeline rodante (solo una o dos épicas siguientes a la actual).
REGLAS DE SINCRONIZACIÓN	La fuente de verdad son los artefactos BMAD (<code>planning_artifacts/</code> , <code>implementation_artifacts/</code>); la herramienta externa es un espejo. Si BMAD diverge, BMAD gana.

Sincronización al inicio (épicas + Epic 1 y 2) y cada vez que se crean historias nuevas (al 80 % de la épica actual). Responsable: BMAD Driver o PO. CÓ PE

Sección "Ops Conventions" en `project-context.md`: tool elegida + estructura épica/historia + responsable de sincronización.

D2 Estrategia de ramas y nomenclatura

RESUELVE	Cómo se gestionan las ramas y los pull requests. Sin acuerdo, los code reviews terminan discutiendo estilo de commits en lugar de código.
MODELO DE RAMAS	<i>Trunk-based con feature branches por historia</i> (recomendado por defecto). Alternativas: <i>epic branches</i> para migraciones destructivas o releases atómicos; <i>release branches</i> para productos con calendario rígido.
GRANULARIDAD PR↔HISTORIA	Una historia, un pull request (recomendado). Alternativas: <i>stacked PRs</i> (3–4 PRs encadenados con herramientas como Graphite); <i>chunked PRs con feature flags</i> . Elegir una y mantenerla.
NOMENCLATURA DE RAMAS	<code><tipo>/<ticket-id>-<slug-corto></code> . Prefijos: <code>feat/</code> , <code>fix/</code> , <code>chore/</code> , <code>refactor/</code> , <code>docs/</code> , <code>test/</code> , <code>perf/</code> . Ejemplo: <code>feat/PROJ-123-add-invoice-pdf</code> .
CONVENCIÓN DE COMMITS	Conventional Commits (mismos prefijos que las ramas). Habilita changelog automático y versionado semántico.
GRANULARIDAD COMMIT	Tres opciones: (a) un commit por historia con WIPs locales que se squashean al PR; (b) un commit por sub-tarea (red, green, refactor, docs) preservando la narrativa; (c) micro-commits con squash al merge.
POLÍTICA DE MERGE	Squash merge, merge commit o rebase. Reviewers: 1 senior para P1–P2; 2 (incluyendo Tech Lead) para P0.
GATES DEL PR	Lint + unit + integración + umbral de cobertura + escaneo de seguridad. Configurados en S2 vía <code>/bmad-testarch-ci</code> ; aquí se confirma cuáles son obligatorios y cuáles informativos.
QUIÉN DECIDE	Tech Lead co-decide con DevOps (la estrategia debe alinearse con CI/CD). QA valida que los gates incluyan testing.
CÓMO PERSISTIRLO	"Ops Conventions" en <code>project-context.md</code> : modelo + nomenclatura + convención + granularidad + merge policy + reviewers por prioridad. Las skills <code>dev-story</code> y <code>code-review</code> leen este contexto para sugerir naming y validar gates.

D3 Flujo de estados del backlog (incluyendo TEA)

RESUELVE	El flujo Agile estándar (<i>To Do</i> → <i>In Progress</i> → <i>Done</i>) oculta los momentos TEA. Sin estados explícitos, ATDD, Test Automation y Code Review no son visibles en el dashboard.
WORKFLOW POR HISTORIA	Ocho estados activos más uno de excepción: <i>Backlog</i> → <i>To Do</i> → <i>ATDD</i> → <i>In Development</i> → <i>Test Expansion</i> → <i>Code Review</i> → <i>Approved</i> → <i>Done</i> . <i>Blocked</i> es estado paralelo con motivo etiquetado (<code>dep:epic-X</code> , <code>infra</code> , <code>external</code> , <code>waiting:design</code>). Caminos de retorno válidos: Code Review puede regresar a In Development o ATDD si revela huecos en los criterios de aceptación; Test Expansion puede regresar a ATDD.
MAPEO SKILL↔ESTADO	Cada estado mapea a una skill BMAD (<code>create-epics-and-stories</code> , <code>sprint-planning</code> , <code>testarch-atdd</code> , <code>dev-story</code> , <code>testarch-automate</code> , <code>code-review</code>).
ADAPTACIÓN SI LA TOOL TIENE MENOS ESTADOS	Si la herramienta solo soporta cinco estados nativos, fusionar ATDD e In Development en "In Progress" con etiquetas <code>stage:atdd</code> o <code>stage:dev</code> ; fusionar Approved y Code Review en "In Review" con etiqueta <code>approved</code> .
WORKFLOW POR ÉPICA	Cinco estados: <i>Planned</i> → <i>In Progress</i> → <i>Stories Done</i> → <i>In TEA Closure</i> → <i>Done</i> . <i>Stories Done</i> es crítico: dispara la transición al cierre TEA y, para la primera épica, es el trigger de S6 facilitada.
VALIDACIÓN RECOMENDADA EN CI	Validador en <code>sprint-status.yaml</code> que bloquea épica → Done sin marcas <code>test-review:done</code> , <code>trace:done</code> , <code>retrospective:done</code> .
QUIÉN DECIDE	QA es obligatorio en esta decisión: sin QA el flujo omite ATDD, Test Expansion y Code Review. PO y Tech Lead aprueban. Las transiciones de retorno se configuran en la tool como parte del output, no como excepciones.
CÓMO PERSISTIRLO	"Ops Conventions" en <code>project-context.md</code> con ambos workflows y el mapping skill↔estado.

D4 Worktrees y multi-agente (opcional — solo si el equipo usa Claude Code)

RESUELVE	Paralelizar trabajo de Claude Code sin colisiones de entorno (puertos, bases de datos, archivos de configuración). Solo aplica si el equipo usa Claude Code; con otros agentes o sin ellos, omitir.
TRES APROXIMACIONES (NO EXCLUYENTES)	① <i>Worktrees</i> (<code>claude --worktree</code>): sesión humana paralela en otra terminal. ② <i>Subagents con isolation: worktree</i> : fan-out dentro de una sesión. ③ <i>Agent teams</i> : orquestación de múltiples sesiones que se mensajan entre sí.
CONFIGURACIÓN WORKTREE-NATIVE	<code>.worktreeinclude</code> en la raíz (lista de archivos ignorados por Git que deben copiarse al worktree: <code>.env</code> , <code>.env.local</code> , <code>secrets</code>); <code>.gitignore</code> añade <code>.claude/worktrees/</code> ; hook <code>WorktreeCreate</code> para puertos parametrizados, <code>COMPOSE_PROJECT_NAME</code> único en Docker o bases de datos con scope.
BULLET OBLIGATORIO EN CLAUDE.MD	When creating worktrees add to <code>.gitignore</code> , cp any env files into this worktree.
/BATCH PARA CODEMODS	Cinco a treinta sub-agentes en worktrees aislados, cada uno como un PR independiente. Cada candidata debe traer receta end-to-end documentada y ser independiente y mergeable por sí sola. Lista versionada (sugerencia: <code>_bmad-output/batch-candidates.md</code>). Anti-candidatos: refactors con dependencias entre archivos, migraciones de schema, cambios con orden estricto. Aprobación: TL o BMAD Driver, no requiere PO.
POLÍTICA DE PRS EN BLOQUE	Reviewer único senior si P2 o P3 y codemod determinístico; auto-merge con etiqueta <code>batch-codemod</code> y CI verde; excepción P0 (auth, financial, security) requiere TL aunque venga de <code>/batch</code> .
CÓMO PERSISTIRLO	"Ops Conventions" en <code>project-context.md</code> — sub-sección Claude Code — + <code>.worktreeinclude</code> + ajuste de <code>.gitignore</code> + bullet en <code>CLAUDE.md</code> + hook opcional <code>WorktreeCreate</code> + lista opcional <code>batch-candidates.md</code> .

7.2 Plantilla resumida de la sección "Ops Conventions"

Al cerrar S4, la sección "Ops Conventions" del archivo `project-context.md` queda con la siguiente forma. La plantilla completa, lista para copiar, vive en el Sec. 10.

```
## Ops Conventions

### D1 – Backlog tool
tool: <Jira|Linear|Notion|...>
estructura: epic-parent + story-children
sync_owner: <BMAD Driver | P0>
sync_cadence: inicio + cada 80% de épica activa

### D2 – Branch & commit strategy
branch_model: <trunk-based|epic-branches|release-branches>
pr_granularity: <1 historia = 1 PR | stacked | chunked-flags>
branch_naming: feat|fix|chore|refactor|docs|test|perf / TICKET / slug
commit_convention: conventional-commits
commit_granularity: <1-por-historia | 1-por-subtarea | micro+squash>
merge_policy: <squash|merge-commit|rebase>
reviewers:
  P0: 2 (incluye Tech Lead)
  P1-P2: 1 senior
  P3: 1 senior

### D3 – State workflow
historia_estados: Backlog,ToDo,ATDD,InDev,TestExp,CodeReview,Approved,Done,Blocked
epica_estados: Planned,InProgress,StoriesDone,InTEAClosure,Done
ci_validator: bloquea épica→Done sin test-review:done + trace:done + retro:done

### D4 – Claude Code (opcional)
modos: [worktrees, subagents-isolation, agent-teams]
worktreeinclude: [.env, .env.local, .secrets/*]
gitignore_addition: .claude/worktrees/
batch_candidates_file: _bmad-output/batch-candidates.md
batch_approval: TL or BMAD Driver
```

8 Sesión S5 – Ciclo iterativo

S5 Ciclo iterativo por historia

Continuo · post-S4

RESUELVE: Implementa todas las historias de cada épica con calidad gobernada por gates, en un ciclo repetible y autónomo. No es sesión facilitada: es el modo de operación del equipo entre S4 y S6.

CUÁNDO SE HACE

Tras S4. Aplica al menos a dos épicas: la primera (configuración e infraestructura) y la segunda en adelante (features de negocio). El cierre de la primera épica es S6 facilitada; el cierre de las épicas posteriores se ejecuta en autonomía.

PARTICIPANTES

- BMAD Driver siempre.
- Resto de roles según el paso del ciclo (ver agenda).

REQUISITOS PREVIOS

- S4 completa con `project-context.md` commiteado.
- Framework de tests y pipeline operativos desde S2.

OUTPUT POR HISTORIA

Pull request mergeado a la rama principal con tests automatizados y revisión de código superada.

CICLO POR HISTORIA — CADENA ESTRICTA

1. `/bmad-create-story` ● — Driver + SM. La primera historia ya viene de S3.
2. `/bmad-testarch-atdd` ● — Driver + QA. Genera tests fallidos (red phase) antes de implementar.
3. `/bmad-dev-story` ● — Driver + Dev (ingeniero asignado, distinto del TL del paso 5). Ciclo red-green-refactor⁹.
4. `/bmad-testarch-automate` ● — Driver + QA. Expande la cobertura tras la implementación.
5. `/bmad-code-review` ● — Driver + Tech Lead. Gate antes del merge.

Las historias del mismo sprint pueden ejecutar el ciclo en paralelo si no hay conflictos de código compartido. Es el paralelismo nativo de Agile.

CIERRE DE ÉPICA — ÉPICAS 2+ EN AUTONOMÍA; ÉPICA 1 = S6 FACILITADA

1. `/bmad-testarch-test-review` ● — puntuación 0–100. Driver + QA.
2. `/bmad-testarch-trace` ● — trazabilidad y decisión de gate. Driver + QA.
3. `/bmad-retrospective` ● — consume las salidas anteriores. Equipo completo.

HITOS NFR DURANTE O TRAS LA IMPLEMENTACIÓN

1. `/bmad-testarch-nfr` en modo **check** — revisión intermedia. Driver + QA + TL + DevOps.
2. `/bmad-testarch-nfr` en modo **gate** — firma final pre-release. Driver + QA + TL + DevOps + PO. FAIL bloquea release.

RECOMENDACIÓN LIDR

Recomendamos planificar al menos dos épicas por proyecto. La épica 1 cubre configuración e infraestructura; la épica 2 en adelante, features de negocio. Forzar features de negocio en la primera épica suele saltarse la configuración del bloque TEA, y los problemas aparecen acumulados en el primer NFR Gate.

Siguiente paso: S6 al cerrar la primera épica feature-completa (todas las historias en Done, código mergeado, sprint cerrado).

9 Sesión S6 — Cierre formal de la primera épica

S6 Cierre formal de la primera épica

2–3 h · post primera épica

9. red-green-refactor: ciclo de Test-Driven Development en tres pasos. Rojo: escribir un test que falle. Verde: implementar lo mínimo para que pase. Refactor: limpiar el código manteniendo el test verde.

RESUELVE: Ejecuta el primer cierre TEA acompañado para que el equipo aprenda el flujo con red de seguridad antes de operar autónomo en épicas posteriores.

CUÁNDO SE HACE

Cuando la primera épica tiene todas sus historias en Done, código mergeado a la rama principal y sprint cerrado. Si quedan historias en In Development o Code Review, S6 espera.

PARTICIPANTES

- BMAD Master **REQ** — Q&A al cierre, ~15–20 min.
- Scrum Master **REQ** — facilita toda la sesión.
- QA / Test Architect **REQ**.
- Product Owner **REQ**.
- Tech Lead **REQ**.
- Devs de la épica 1 **REQ**.
- DevOps **REQ** — lidera el toolkit post-S6 si aplica.
- BMAD Driver **REQ**.

REQUISITOS PREVIOS

- Primera épica feature-completa con todas las historias en Done.

OUTPUT

Test Review report + matriz de trazabilidad con decisión de gate + retrospectiva con lecciones y action items + confirmación explícita de autonomía del equipo para épicas posteriores.

AGENDA — ORDEN ESTRICTO, NO PARALELIZABLE

1. **Apertura del SM** — confirma que la épica está feature-completa y presenta la agenda.
2. **/bmad-testarch-test-review** ● — Driver ejecuta; QA interpreta. Input: suite acumulado.
3. **/bmad-testarch-trace** ● — Driver + QA. Consume la puntuación del paso anterior.
4. **/bmad-retrospective** ● — equipo completo. Consume las salidas de los pasos 2 y 3.
5. **Incorporación del Master: Q&A operativo** ~15–20 min — resuelve dudas del primer cierre y confirma autonomía para épicas posteriores.
6. **Cierre del Master** con confirmación explícita de autonomía. Si el equipo no está listo, documenta plan de refuerzo: lectura del Libro 5, mini-formación asíncrona o nueva sesión facilitada para la épica 2.

Siguiente paso: operación autónoma del equipo en épicas posteriores. La siguiente sesión facilitada típica suele ser de análisis de métricas (fuera del alcance de este libro).

TOOLKIT POST-S6 · ASÍNCRONO · SIN SESIÓN NUEVA

NFR CHECK DIDÁCTICO

NFR es skill de hito o release, no de cierre de épica. Cómo activarla tras S6 depende del contenido de la primera épica:

- **La épica 1 incluyó código con impacto NFR** (no es configuración o infraestructura puras) → **/bmad-testarch-nfr** en modo *check*, asíncrono y sin facilitador. Ejecutan QA + TL + DevOps. La salida queda archivada como plantilla para futuros checks y gates.
- **La épica 1 fue configuración o infraestructura puras** (caso típico) → diferir hasta el cierre de la épica 2, primera con features productivas. Documentar la decisión en el documento de retro.
- **Salida ambigua o evidencia insuficiente** → consultar a **/bmad-tea** (Murat). Si no se resuelve por chat, mini-sesión facilitada de 45–60 min con el Master.

RECOMENDACIÓN LIDR

Reservar al menos una hora tras S6 para que QA, TL y DevOps ejecuten juntos el primer NFR Check. Es la última red de seguridad antes de afrontar TEA solos en las épicas posteriores.

10 Plantillas operativas

Recursos imprimibles para uso del equipo. Las plantillas son nuestras — reflejan el formato que usamos en adopciones reales. Pueden adaptarse al contexto de cada proyecto.

10.1 Plantilla completa de `project-context.md`

Estructura recomendada del archivo. Las skills `create-story`, `dev-story`, `sprint-status`, `code-review` y varias TEA leen este archivo como contexto cuando se invocan en una nueva conversación. La sección "Ops Conventions" se completa en S4.


```

# project-context.md

## Identidad del proyecto
project_name: "<nombre>"
project_type: "<web|api|cli|mobile|library>"
stack: "<tecnologías principales>"
languages: "<es|en|...>"
team_size: "<número>"

## Reglas críticas
# Reglas que el agente debe respetar siempre. Una por línea.
- "todas las funciones públicas exportadas llevan tipos explícitos"
- "los errores se propagan, no se silencian"
- "los tests usan el helper testdb, no el helper memdb"

## Patrones del proyecto
# Patrones de código aceptados, con un ejemplo de cada uno.
- nombre: "ejemplo-X"
  cuándo: "... "
  ejemplo: "... "

## Ops Conventions

### D1 – Backlog tool
tool: <Jira|Linear|Notion|GitHub Projects|...>
estructura: epic-parent + story-children
sync_owner: <BMAD Driver|P0>
sync_cadence: inicio + cada 80% de épica activa

### D2 – Branch & commit strategy
branch_model: <trunk-based|epic-branches|release-branches>
pr_granularity: <1 historia = 1 PR | stacked | chunked-flags>
branch_naming: "<tipo>/<ticket-id>--<slug-corto>"
prefijos: [feat, fix, chore, refactor, docs, test, perf]
commit_convention: conventional-commits
commit_granularity: <1-por-historia|1-por-subtarea|micro+squash>
merge_policy: <squash|merge-commit|rebase>
reviewers:
  P0: "2 (incluye Tech Lead)"
  P1-P2: "1 senior"
  P3: "1 senior"
pr_gates:
  obligatorios: [lint, unit, integration, coverage-threshold]
  informativos: [security-scan]

### D3 – State workflow
historia_estados:
  - Backlog
  - To Do
  - ATDD # ↔ /bmad-testarch-atdd
  - In Development # ↔ /bmad-dev-story
  - Test Expansion # ↔ /bmad-testarch-automate
  - Code Review # ↔ /bmad-code-review
  - Approved
  - Done
  - Blocked # estado paralelo, motivo etiquetado
epica_estados:
  - Planned
  - In Progress
  - Stories Done
  - In TEA Closure
  - Done
ci_validator: |
  bloquea épica→Done sin marcas:

```